

Universidad Tecnológica Nacional – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ingeniería y Calidad de Software

Trabajo Práctico Evaluable N.º 6

TDD – Desarrollo guiado por pruebas

User Story: Inscribirme a actividad (EcoHarmony Park) · **Documento de decisiones de diseño**

Año: 2026
Docentes: Ing. Laura Covaro, Ing. Cecilia Massano, Ing. Constanza Garnero, Ezequiel Izaguirre
Curso: 4K3
Grupo Nro: 8
Integrantes:

- ULIANA, Agustin – 97828
- LEDO FLORES, Francisco – 86489
- CASTELO, Matias – 88413
- DIAZ, Ivan – 87473
- SORUCO, Jonatan – 79456
- GIAIMO, Gino – 78426
- PÁEZ DE LA TORRE, Matias – 83289
- RAMALLO, Mateo – 94441
- CORREA, Jeremías – 88714
- VARGAS FERNANDEZ, Rodrigo – 96417
- FORTI, Bruno – 80818
- TRONCOZO, Valentina – 86268
- LEYES MOLINA, Selene – 65743
- CARDOZO ROJAS, Fernando – 89064

Fecha de Entrega: 21/10

Documentos hermanos: plan de pruebas · preguntas al PO · partición de equivalencias · código

> COMO visitante QUIERO inscribirme a una actividad PARA reservar mi lugar en la misma.

La consigna del práctico pide, además del código y de las pruebas, un *documento que justifique las decisiones de diseño tomadas*. Este es ese documento. Cada decisión se presenta con el mismo formato: **qué se decidió**, **por qué**, **qué alternativa se descartó** y **qué prueba lo respalda**, para que se pueda verificar en el código.

Índice

1. Resumen
2. Cómo se aplicó TDD
3. Arquitectura
4. Decisiones de diseño
5. Estilo de código: PEP 8
6. Tecnologías utilizadas y por qué
7. Trazabilidad con la User Story
8. Estrategia de pruebas
9. Límites y trabajo futuro
10. Cómo ejecutarlo
11. Bibliografía

1. Resumen

Tema	Resultado
Lenguaje y estilo	Python 3.13 (backend; compatible desde 3.11) y JavaScript/React 19 (front). **PEP 8**, verificado automáticamente con `ruff` (largo de línea 100).
Enfoque	TDD de Kent Beck: lista de pruebas, y luego un ciclo **Red** → **Green** → **Refactor** por cada comportamiento, visible en `git log`.
Pruebas automatizadas	**380 del backend** (345 sin base de datos + 35 contra PostgreSQL real y `qrcode` real) y **110 del front**. Todas verdes.
Persistencia	PostgreSQL 16 en Docker, SQL plano (sin ORM), visible desde DBeaver.
Piezas reales y gratuitas	QR con la librería `qrcode`; email real por SMTP con Gmail; PostgreSQL local.
Interfaz	React + Vite, paleta y tipografía (Lato) del PO, estilo *Liquid Glass*, responsive.

Todo lo que el PO puede cambiar (horarios, cupos, edades, talles, máximo por inscripción) es **configuración inyectada**, no lógica fija. Las respuestas del PO se absorbieron cambiando datos, no reescribiendo pruebas.

2. Cómo se aplicó TDD

2.1 Proceso

1. **Test List** (plan de pruebas): antes de escribir código se listaron las pruebas, de la más simple a la más compleja, y se enviaron las dudas al PO. La lista no se consideró cerrada: las pruebas que fueron apareciendo se anotaron en ella antes de escribirse.

2. **Un ciclo por comportamiento**, siempre en el mismo orden y con un commit por cada etapa:

    - `test(ID): rojo` – la prueba nueva **falla por la razón correcta** (se verifica antes de commitear).
    - `feat(ID): verde` – el código mínimo para que pase (*Fake It*, *Triangulate* u *Obvious Implementation*).
    - `refactor(...)` – mejora del diseño sin agregar comportamiento y con todo en verde. 3. **Historial agrupado.** Para que el historial sea legible no hay un commit por cada prueba: se agrupan las pruebas de un mismo tema en un commit rojo y un commit verde (*Clean Check-in* de Beck). Cuando una prueba pasa de entrada se lo dice en el mensaje (`ya verde`). 4. **Prueba de aceptación primero** (ATDD, Ble cap. 3): `ACE-01` se escribió antes que el servicio, marcada como *fallo esperado*, y se activó cuando las piezas estuvieron.

2.2 Patrones de Beck usados y dónde se ven

Patrón	Dónde
Starter Test	Primer ciclo: `Actividad` con cupo 10 y pedido de 3 → quedan 7 (`ACT-01`).
Fake It / *Triangulate*	Cupos (`ACT`) y edades mínimas: se parte de una constante y se generaliza al agregar un segundo caso.

Patrón	Dónde
Obvious Implementation	Validaciones simples de `Visitante` (DNI, edad, nombre).
Child Test	Las reglas combinadas de `Inscripcion` se descompusieron en pruebas chicas, una por regla (`INS-01..20`), en lugar de una prueba grande.
Mock Object	`ServicioEmail` (acción): se verifica que se envió una vez con el mensaje correcto.
Crash Test Dummy	Dobles que lanzan excepción: email caído, QR roto, repositorio caído (`SRV-15..17`).
Clean Check-in	Nada se commitea en rojo salvo la prueba nueva y nada se commitea sin ejecutar la suite.
Assert First y *Evident Data*	Las pruebas empiezan por el `assert` ; los datos se leen a simple vista (cupo 10, piden 3).

2.3 Política de dobles de prueba (Ble, cap. 6)

Colaborador	Tipo	Doble en unitarias
`Reloj` , `Parque` , lecturas del repositorio, `GeneradorQR`	consulta	*stub / fake*: se verifica el **estado**, no cuántas veces se llamó
`ServicioEmail`	acción	*mock*: se verifica que se envió, una vez, con el mensaje correcto
Repositorios	almacenamiento	*fake* en memoria **validado con las mismas pruebas de contrato que el real**
Fallas (email, QR, base)	error improbable	*Crash Test Dummy*

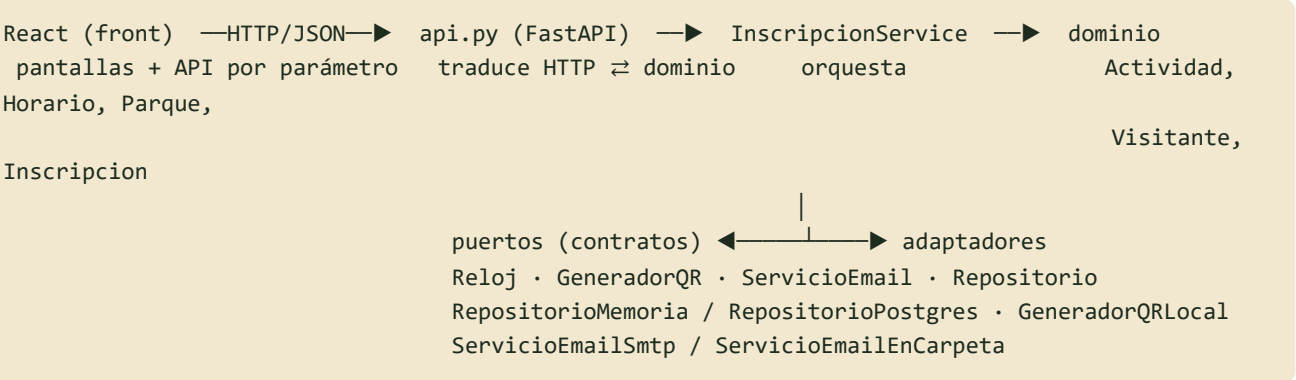
Por qué. Verificar llamadas a una consulta vuelve la prueba frágil (se rompe ante un refactor que no cambia el comportamiento). Y un fake que no se contrasta con el real termina mintiendo: por eso existe el grupo de **pruebas de contrato**, que se ejecutan dos veces, contra el fake y contra PostgreSQL (§8.2).

2.4 Categorías de pruebas

Cat.	Qué usa	Cuántas	Cuándo corre
U – unitarias	dominio y dobles; sin red, disco ni reloj real	345 (backend) + 110 (front)	siempre (`pytest` , `vitest`)
C – contrato / integración local	PostgreSQL local, `qrcode`	35	con la base levantada (marcadores `db` y `local`); sin base se saltean con un aviso
X – externas	email real a una casilla	1 (se pide con `pytest -m externa`)	manual, antes de la entrega (marcador `externa`)

3. Arquitectura

3.1 Capas



Capa	Módulos (`backend/src/ecoharmony/`)	Responsabilidad
Dominio	`actividad`, `horario`, `parque`, `visitante`, `inscripcion`, `motivos`, `terminos`, `terminos_texto`, `catalogo`, `usuarios`, `contrasenas`	Reglas del negocio. Sin I/O.
Aplicación	`inscripcion_service`, `disponibilidad`, `mensaje_email`, `mensaje_html`, `qr`	Orquesta el caso de uso; armar el mensaje (texto y HTML) y el contenido del QR son funciones puras.
Puertos	`puertos`	Contratos (`Protocol`) de reloj, QR, email y repositorios.
Adaptadores	`recursos/` (logo del email), `repositorio_memoria`, `repositorio_postgres`, `repositorio_usuarios_*`, `generador_qr_local`, `servicio_email_smtp`, `servicio_email_en_carpeta`, `reloj`	Lo que toca el mundo exterior.
Entrada	`api`, `main`, `preparar_base`, `entorno`	HTTP, armado de dependencias, configuración.

Decisión: dependencias hacia adentro e inyección por constructor. `InscripcionService` recibe repositorio, parque, reloj, generador de QR y servicio de email. Nada del dominio importa un adaptador.
Por qué: es lo que permite el TDD sin infraestructura (Ble: dobles) y reemplazar un doble por la pieza real sin tocar `Inscripcion` ni el servicio. **Descartado:** instanciar los colaboradores dentro de las clases (haría imposible simular la hora, un email caído o una base caída).

3.2 Modelo del dominio

Clase	Rol
`Actividad`	Nombre, cupo por turno, edad mínima, si requiere talla y sus T&C. Dueña de la regla de cupos.
`Horario`	Fecha + hora de inicio de un turno de una hora.
`Parque`	Días y horas de funcionamiento (lunes, 25/12 y 1/1 cerrado; 8:30 a 19:00).
`Visitante`	Nombre, DNI, edad y talla; se valida a sí mismo contra la actividad.

Clase	Rol
`Inscripcion`	Reúne titular, actividad, horario, visitantes y T&C aceptados; aplica las reglas combinadas.
`Usuario`	Email y contraseña; el visitante debe estar registrado (decisión del grupo).
`TerminosYCondiciones`	Actividad, versión y texto.
`MotivoRechazo`	Enumeración de las razones por las que se rechaza (D1).

4. Decisiones de diseño

D1. Rechazar con un motivo, no con un booleano

Se decidió: `Inscripcion` y el servicio devuelven una lista de `MotivoRechazo` (`SIN_CUPO`, `DNI_INVALIDO`, `TALLA_REQUERIDA`, `PARQUE_CERRADO`, `FUERA_DE_VENTANA`, `TERMINOS_NO_ACEPTADOS`, `DNI_YA_INSCRIPTO`, `USUARIO_NO_REGISTRADO`, …) y, para los visitantes, un diccionario *posición → motivos*.
Por qué: una prueba que solo mira `False` no distingue un rechazo correcto de uno accidental. Con el motivo, cada prueba verifica *la razón*. Además es lo que permite al front mostrar "Visitante 1: el DNI debe tener 7 u 8 dígitos".
Descartado: lanzar una excepción por regla (obliga a atrapar de a una y se pierden los demás errores) o devolver un texto (las pruebas quedarían atadas al idioma).
Respaldo: `INS-01..20`, `VIS-01..18`, `SRV-*`, `MSJ-01..07` (front).

D2. Todas las reglas del PO como configuración inyectada

Se decidió: cupos, edades mínimas, tallas, máximo de 10 personas, ventana de 24 h a 15 min y horarios del parque son datos (constantes con nombre, `catalogo.py`, constructor de `Parque`), no condiciones sueltas en la lógica.
Por qué: el PO ya cambió respuestas durante el trabajo y el grupo podía equivocarse en una; cambia un dato, no una prueba. Los textos de T&C también se arman con esos datos (D12), de modo que no hay números repetidos que se desactualicen.
Respaldo: `ACT-*`, `PAR-*`, `HOR-*`, `CAT-*`.

D3. El tiempo es un colaborador (`Reloj`)

Se decidió: el servicio recibe un `Reloj` y nunca consulta la hora del sistema.
Por qué: "ya comenzó", "falta menos de 15 minutos" y "más de 24 horas" no se pueden probar con el reloj real. Las pruebas fijan el reloj en el miércoles 2026-11-04 08:00.
Respaldo: `HOR-*`, `INS-*`, `test_reloj.py`.

D4. Un QR por visitante, en un único email

Se decidió: el contenido del QR se arma con una función pura (`contenido_del_qr`: código de reserva y número de visitante, por ejemplo `INS-0007|V2`) y se genera un PNG por visitante, todos en **un** email (a la vista en el cuerpo y también adjuntos).
Por qué: lo confirmó el PO (cada persona entra con su propio QR); un solo email evita que el titular reciba un mensaje por visitante. Separar *qué dice el QR* de *cómo se dibuja* permite probar el contenido sin imagen y reemplazar la librería sin tocar nada más.

Respaldo: `QR-01..08` (el contrato se ejecuta con el generador falso y con `qrcode`, que devuelve un PNG válido y distinto por contenido).

D5. Reserva de cupo atómica en SQL y restricción de unicidad en la base

Se decidió: el cupo se reserva con **una sola sentencia** `INSERT … ON CONFLICT DO UPDATE … WHERE ocupados + n <= cupo`, y el conflicto de horario por DNI se refuerza con `UNIQUE (dni, fecha, hora)`. El servicio traduce la carrera a `DNI_YA_INSCRIPTO`.
Por qué: "leer el cupo, comparar y escribir" son tres pasos: dos pedidos simultáneos por el último lugar ganarían los dos. La comprobación previa del servicio (`SRV-13`) da el mensaje amable; la restricción de la base es la garantía real.
Descartado: bloquear en la aplicación (no sirve con más de un proceso) y confiar solo en la comprobación previa.
Respaldo: `REP-05` (dos hilos disputando el último lugar contra PostgreSQL real), `REP-09`, `SRV-19`, `SRV-20`.

D6. Orden de las operaciones al inscribir

Se decidió: validar → generar el código de reserva → **generar los QR** → **guardar** (reserva de cupo + inscripción) → **enviar el email con reintentos**. Si el email no sale después de 3 intentos, se **anula** la inscripción (libera cupo y DNI) y se informa el error (503).
Por qué:

- El PO dijo que el email no debería fallar y el grupo decidió que, si falla, **la inscripción no queda confirmada** (P13b). Reintentar antes de anular cubre las caídas transitorias.
- Se generan los QR *antes* de guardar para que un QR roto no deje una inscripción a medias (`SRV-16`).

Limitación conocida (honestidad del diseño): si el proceso se cae justo entre guardar y enviar, la inscripción quedaría guardada sin email. La mejora natural es un estado *pendiente* con reintentos en segundo plano; se dejó fuera del alcance de este TP.
Respaldo: `SRV-01..17`.

D7. Validar sin efectos (`POST /inscripciones/validar`)

Se decidió: el servicio tiene `validar` además de `inscribir`: aplica las mismas reglas pero **no guarda, no reserva cupo ni envía nada**. La API lo expone y el paso 3 del front lo usa al tocar *Continuar*.
Por qué: el grupo pidió que los errores de los visitantes (por ejemplo "Visitante 1: el DNI debe tener 7 u 8 dígitos, la edad debe estar entre 0 y 100") aparezcan en el paso donde se cargan y no recién en la confirmación. La alternativa obvia, repetir las reglas en JavaScript, tiene dos fuentes de verdad que terminan divergiendo. Con `validar`, **las reglas viven en un solo lugar** y el front solo traduce motivos a mensajes y campos.
Efecto colateral útil: el mismo chequeo incluye cupo y DNI ya inscriptos, así que esos avisos también llegan antes.
Respaldo: `SRV-24..30`, `API-14..17`, `VAL-F01..14`, `CAM-01..04`, `APP-10..12`.

D8. Persistencia: PostgreSQL con SQL plano

Se decidió: PostgreSQL 16 (Docker, puerto 5433), `psycopg` 3 y SQL escrito a mano (`sql/schema.sql`), sin ORM. Dos bases: `ecoharmony` (desarrollo) y `ecoharmony_test` (pruebas). El esquema se escribe con `CREATE … IF NOT EXISTS`, de modo que `preparar_base` se puede volver a ejecutar.

Por qué: el grupo quería abrir la base en DBeaver; el SQL plano se ve y se entiende (el grupo eligió "el SQL más simple" y PEP 8). El esquema reaplicable permite agregar tablas nuevas a una base ya existente sin perder datos (`PREP-03`).
Descartado: SQLite (no se ve igual en DBeaver ni tiene el mismo manejo de concurrencia) y un ORM (oculta justo la sentencia atómica de D5).
Respaldo: `test_esquema_postgres.py` , `test_preparar_base.py` , `REP-*` .

D9. Fake en memoria + prueba de contrato

Se decidió: el mismo conjunto de pruebas (`REP-01..09`) se ejecuta contra `RepositorioMemoria` y contra `RepositorioPostgres` .
Por qué: el fake sirve para que las unitarias sean rápidas, pero solo si se comporta como el real. **La prueba de contrato ya encontró un defecto real:** en el fake, `guardar` reservaba el cupo sobre la *copia* de la actividad que traía la inscripción y no sobre la del repositorio (`REP-03`). Sin el contrato, el fake habría mentido.

D10. Usuarios registrados y sesión simple

Se decidió: el visitante debe registrarse con **email + contraseña**; el email de la inscripción es el del usuario (P01/P02). Contraseñas con `hashlib.scrypt` y sal (biblioteca estándar); sesión como token firmado con HMAC-SHA256 que vence a las 8 horas. Email y contraseña incorrectos fallan con el mismo mensaje.
Por qué: la US no decía a qué email enviar; unir inscripción con usuario resuelve la pregunta y evita pedir un email por cada inscripción. Se usó solo la biblioteca estándar para no sumar dependencias por algo simple.
Privacidad: sin sesión, el rechazo es siempre `USUARIO_NO_REGISTRADO` y **no** revela si un DNI ya estaba inscripto (`SRV-23` , hallazgo de la prueba manual).
Descartado: guardar la contraseña con un hash rápido (SHA) o en texto plano; JWT con librería externa.
Respaldo: `USU-01..09` , `INS-20` , `SRV-21` , `SRV-23` , `ACE-08` .

D11. Email: Gmail por SMTP y un reemplazo para desarrollo

Se decidió: `ServicioEmailSmtp` envía por SMTP con STARTTLS y un **tiempo límite de 10 s**, configurado desde el `.env` . El grupo eligió **Gmail** (contraseña de aplicación): es gratuito, el mensaje sale de una casilla real y llega a la bandeja de entrada sin verificar dominios. Si el `.env` no tiene SMTP, la API arranca con `ServicioEmailEnCarpeta` , que guarda el texto y los QR en `salida_de_emails/` (ignorada por git). **El mensaje:** HTML con la marca (paleta del PO, Lato, el logo, y por cada visitante su nombre, su DNI y su código QR), con una versión en texto plano y los QR también adjuntos. Las imágenes viajan dentro del propio mensaje (`cid:`), así se ven sin cargar imágenes remotas, y todo dato escrito por el usuario se escapa antes de entrar al HTML. Lleva `Date` y un `Message-ID` único, cabeceras que los filtros esperan para no tratarlo como correo basura. El **asunto es único por inscripción** (código, actividad, fecha y horario): con el mismo asunto, Gmail junta en una sola conversación los mails de dos inscripciones distintas y parece que llegó uno solo. **Por qué:** el PO exige un email real, pero las pruebas automáticas no pueden depender de internet ni de credenciales. La carpeta de desarrollo permite probar todo el recorrido sin enviar nada, y el adaptador sigue sirviendo para otro proveedor cambiando solo el `.env` . **Descartado:** Brevo u otro servicio transaccional gratuito (piden verificar remitente o dominio; sin eso suele ir a spam) y guardar la clave en el código. **Hallazgos de las pruebas manuales:** (1) con credenciales SMTP de ejemplo en el `.env` , la API se quedaba colgada intentando conectarse: se agregó el tiempo límite (`SMTP-06`) y el `.env.example` trae el SMTP comentado; (2) la API lee el `.env` solo al arrancar, así que si ya estaba corriendo sigue guardando los emails en la carpeta: hay que reiniciarla al cambiar el `.env` ; (3) dos inscripciones distintas se veían como un solo mail por el asunto repetido (`EML-19/20`). **Prueba externa (categoría X):** `test_email_externo.py` (marcador `externa`) envía por

Gmail el mismo email que arma el sistema, con dos QR, a `EMAIL_DE_PRUEBA`. Queda fuera de la corrida normal (`addopts = -m 'not externa'`) y se saltea sin credenciales. **Spam:** no se puede garantizar la bandeja de entrada (depende de la reputación del remitente); lo que sí se controla es enviar un mensaje bien formado (HTML con texto alternativo, `Date`, `Message-ID`, remitente igual al usuario SMTP). El primer correo de prueba, de texto plano, cayó en spam; marcarlo «No es spam» la primera vez ayuda. **Respaldo:** `SMTP-01..13`, `MAIL-01/02`, `EML-01..20` y la prueba externa.

D12. Términos y condiciones propios de cada actividad

Se decidió: un texto distinto por actividad, redactado por `redactar_terminos(nombre, edad_minima, requiere_talla, cupo_por_turno)` en `terminos_texto.py`. Todos siguen el mismo modelo de **11 apartados** (el que propuso el grupo): 1 objeto de la actividad, 2 requisitos de inscripción, 3 normas de seguridad, 4 equipo o materiales, 5 salud y condiciones de participación, 6 responsabilidad de la organización, 7 modificaciones, suspensión o cancelación, 8 cuidado de las instalaciones y del medioambiente, 9 datos personales, 10 fotografías y videos, 11 aceptación. **Qué cambia en cada actividad:** el objeto, las normas de seguridad, el cuarto apartado, la salud y el cuidado del entorno.

Actividad	Cuarto apartado	Normas y salud propias (resumen)
Tirolesa	Uso del equipo de seguridad	Línea de vida, arnés, casco y polea que coloca y revisa el personal; esperar la autorización antes de cada descenso; sin objetos sueltos; calzado cerrado; avisar problemas de espalda, cuello o corazón, epilepsia, vértigo, lesiones o embarazo.
Safari	Contacto con los animales y alimentación	Seguir al guía y no cruzar vallas; no alimentar, tocar ni molestar a los animales; voz baja y sin flash; solo el personal alimenta; avisar alergias y síntomas contagiosos.
Palestra	Uso del equipo de seguridad	Pared de escalada con asegurador; arnés y casco; no escalar sin autorización ni descalzo; no pararse en la zona de caída; avisar lesiones, vértigo o embarazo.
Jardinería y cuidado de espacios verdes	Uso de herramientas y materiales	Herramientas y fertilizantes solo con indicaciones del personal; guantes; calzado cerrado; avisar alergias a plantas, polen o tierra; protector solar.

Qué es fijo y qué sale de los datos: los números (edad mínima, talla, cupo por turno, 24 horas, 15 minutos, máximo de 10 personas) salen de los mismos datos que usa el dominio, de modo que el texto no puede decir "8 años" si la regla dice 10. Una actividad sin texto propio **no se puede crear** (error). Los textos se guardan con una versión interna (`v1`), que no se muestra al visitante, y se aceptan por actividad.
Cómo se redactaron: se tomó como base el modelo de términos y condiciones de participación que aportó el grupo y se adaptó cada actividad con las prácticas habituales que publican parques de aventura (tirolesa), reglamentos de rocódromos (palestra), parques zoológicos y de safari (safari) y guías de seguridad de huertas y talleres (jardinería); las fuentes están en la bibliografía. No existe una norma única para estas actividades: los parques fijan sus propios requisitos. **Menores de edad:** el texto indica que la inscripción y la autorización de menores deben hacerlas su madre, padre, tutor o representante legal; al aceptar, quien se inscribe declara serlo. El sistema **no verifica** esa condición (solo exige un usuario registrado): es una declaración, no un control técnico. **Firma:** el modelo original termina con campos de firma; en un sistema web se reemplazan por la casilla «Acepto los términos y condiciones» de la inscripción (`TERMINOS_NO_ACEPTADOS` si no se marca). **Importante:** el contenido es un **borrador redactado por el grupo**; el PO no lo valida. **Respaldo:** `TYC-01..32` (estructura de 11 apartados en orden, normas propias, equipo, salud, menores, fotografías, aceptación sin firma, reglas del PO y textos distintos entre sí), `CAT-*`, `CONF-F*`.

D13. Interfaz (front)

Decisión	Por qué
API por parámetro: cada pantalla recibe el cliente HTTP y se prueba con una API falsa (`apiFalsa.js`)	Mismo principio que el backend; las pruebas no dependen de red.
Vitest + React Testing Library en lugar de Jest	Vitest es el ejecutor natural de Vite, con la misma API que Jest. *Cambio respecto de lo planeado, avisado al grupo.*
Liquid Glass con `liquid-glass-react`, encerrada en un componente (`Vidrio.jsx`)	La exigencia es estética; si la librería falla o se reemplaza, se cambia un archivo. La paleta (`386641 · 6A994E · A7C957 · F2E8CF · BC4749`) y Lato las controla el CSS.
Límites de la librería	Se centra sola (rehecho como capa detrás del contenido) y su mapa de refracción es cuadrado: solo se usa en elementos con proporción ≤ 1,6 (`VID-11/12`); el resto usa el vidrio esmerilado del CSS. En Safari y Firefox la refracción no se ve y queda el esmerilado.
Accesibilidad	`prefers-reduced-transparency` (panel sólido), `prefers-reduced-motion`, foco visible, objetivos táctiles de 46 px, `aria-invalid` en los campos con error.
Calendario del PO en el front	Solo se pueden elegir hoy y mañana (ventana de 24 h); lunes, 25/12 y 1/1 aparecen cerrados. La regla real sigue en el backend.
Sesión en `localStorage` con `try/catch`	Conserva el acceso al recargar; si el navegador lo bloquea, la app funciona igual.
Mensajes en español por motivo (`mensajes.js`)	La API devuelve motivos, no textos (D1).

D14. Estilo de código y herramientas

Se decidió: el estilo de código del grupo es **PEP 8** y se verifica automáticamente con `ruff`; el detalle de las reglas y de cómo se cumplen está en la sección 5. **Por qué:** la consigna pide usar el documento de estilos elegido en el TP2. Una regla que solo está escrita se olvida; una que se verifica en cada corrida se cumple.

D15. Flujo de trabajo en git

Una rama por tema (`feature/…`, `docs/…`), integradas a `main` con *pull request* y commit de merge. Los commits siguen `tipo(ámbito): rojo|verde|refactor – qué`. El historial se agrupa por tema (§2.1) para que cada ciclo se lea sin ruido.

D16. Identidad de marca: logo del grupo

Se decidió: usar el logo que eligió el grupo (emblema circular con hojas, ciervo, ave, sol y agua, y el nombre «EcoHarmony Park»). Se entrega como PNG con **fondo transparente** (`codigo/frontend/public/logo.png`, versión completa, y `logo-emblema.png`, solo el emblema). El emblema está en el encabezado de todas las pantallas (también sin sesión) y como icono de la pestaña; el logo completo, en la portada de este documento. **Fondo:** el original tenía un fondo blanco opaco. Se lo quitó (no se recortó a mano: se calculó la transparencia de cada píxel y se des-mezcló el color con el fondo, para que los bordes no queden con un halo blanco). Se verificó sobre crema, blanco y verde oscuro: sobre claro se ve nítido, **pero el texto y las hojas oscuras no se leen sobre el verde oscuro** de la página, por eso el logo solo se usa sobre superficies claras.

Colores en el sistema: el encabezado es un panel de vidrio translúcido, y debajo asoman los verdes de la página, que apagaban las hojas del logo. Por eso el emblema lleva un círculo de fondo claro propio (#f8f9f3) con una sombra suave y los colores se ven como en el original.
Número de grupo: la consigna y las respuestas del PO de este módulo no piden que el logo lleve el número de grupo, por lo que no se incluye.
Límite: al ser una imagen, no escala sin perder nitidez como un SVG; el emblema se entrega a 360 px (6 veces el tamaño con que se muestra) y el logo completo a 760 px.
Respaldo: `LOGO-01` (el encabezado muestra el emblema con la descripción «Logo de EcoHarmony Park»), `LOGO-02` (sigue con la sesión iniciada).

5. Estilo de código: PEP 8

La consigna pide implementar con el lenguaje y **el documento de estilos de código elegidos por el grupo para el TP2**. El grupo eligió **PEP 8** (*Style Guide for Python Code*) para el backend. Esta sección resume qué reglas se aplican y cómo se comprueba que se cumplen.

5.1 Reglas aplicadas

Tema	Regla (PEP 8)	Ejemplo en este proyecto
Sangría	4 espacios por nivel, sin tabuladores	todo el código
Largo de línea	PEP 8 fija 79 y admite que el equipo acuerde hasta 99–100: el grupo acordó **100**	`line-length = 100` en `pyproject.toml`
Nombres de funciones, métodos y variables	`snake_case`	`crear_actividades`, `redactar_terminos`, `cupo_por_turno`
Nombres de clases	`CapWords`	`InscripcionService`, `RepositorioPostgres`, `MotivoRechazo`
Constantes	`MAYUSCULAS_CON_GUIONES_BAJOS`	`LARGO_MINIMO_DE_CONTRASENA`, `MAXIMO_DE_PERSONAS`
Miembros no públicos	prefijo `_`	`_evaluar`, `_requisitos`, `_DE_LA_ACTIVIDAD`
Importaciones	al principio, una por línea, agrupadas (biblioteca estándar, terceros, propias) y ordenadas	verificado con la regla `I`
Espacios en blanco	dos líneas entre definiciones de nivel superior, una entre métodos; espacios alrededor de operadores	formateo automático con `ruff format`
Comparaciones	`is None` / `is not None`, nada de `== None`; sin `import *`	regla `F`
Documentación	*docstrings* en módulos y en clases o funciones cuyo propósito no es evidente	`terminos_texto.py`, `Inscripcion`, `InscripcionService.validar`

Convención propia del grupo (permitida por PEP 8, que sugiere el idioma del equipo): identificadores y nombres de pruebas en **español, sin tildes ni eñes** (`test_el_ultimo_cupo_lo_obtiene_solo_la_primera_inscripcion`). Los textos que ve el usuario sí llevan tildes. Cada prueba se nombra por el comportamiento y se lee como una frase.

5.2 Cómo se verifica

`ruff` (verificador y formateador) con las reglas **E** y **W** (pycodestyle, es decir PEP 8), **F** (errores como nombres sin usar), **N** (`pep8-naming` , nombres) e **I** (orden de importaciones):

```
python -m ruff check .     # All checks passed!
python -m ruff format .    # formatea
```

El resultado de la corrida final del código entregado es **0 infracciones**. Además, `ruff` se ejecutó en cada paso del ciclo TDD (antes de cada commit), de modo que el estilo no es una limpieza de último momento. *Ejemplo real:* al agregar los términos y condiciones la herramienta marcó líneas de más de 100 caracteres (E501); se corrigieron en un commit de *refactor* sin cambiar el comportamiento (`refactor(T&C): …`).

5.3 El front (JavaScript)

PEP 8 es una guía de Python y no aplica a JavaScript. En el front el grupo mantuvo convenciones propias y homogéneas: componentes de React en `PascalCase` (`PantallaHorario.jsx`), funciones y variables en `camelCase` , constantes en `MAYUSCULAS` , una responsabilidad por archivo, sangría de 2 espacios y comillas dobles. **No se configuró un verificador automático (ESLint/Prettier) para el front**: es una limitación reconocida; el estilo se mantuvo por consistencia y revisión.

6. Tecnologías utilizadas y por qué

La consigna exige **un lenguaje elegido por el grupo, pruebas automatizadas, el ciclo Red-Green-Refactor, la implementación funcional de la User Story y este documento**. No exige interfaz gráfica, base de datos ni servidor. Cada tecnología de la tabla se evaluó con una pregunta: *¿es lo que pide la consigna, la User Story o el PO, o es una decisión del grupo?* y, en ese caso, *¿qué aporta y cuánto cuesta?*

Tecnología	Para qué se usa	Origen	Qué aporta / costo
Python 3.13	Backend y reglas del negocio	Consigna: lenguaje a elección	Legible y con buen ecosistema de pruebas; el grupo ya tenía su guía de estilo (PEP 8). Se desarrolló con 3.13; el código es compatible desde 3.11.
pytest 8	Ejecutar las pruebas	Consigna: pruebas automatizadas (herramienta libre)	Parametrización (un caso por actividad), *fixtures* y marcadores (`db` , `local` , `externa`).
ruff	Verificar y formatear según PEP 8	Consigna: estilo del TP2	Automatiza el cumplimiento del estilo; una herramienta sola reemplaza varias.
qrcode[pil]	Generar el código QR (PNG)	**User Story**: el email debe contener un QR	Real, gratuito y local; el contrato de pruebas asegura que sea reemplazable.
`smtplib` (biblioteca estándar)	Enviar el email por SMTP	**User Story / PO**: correo real	Sin dependencia nueva; se usa con Gmail y sirve para otro proveedor con solo cambiar el `.env` .
`hashlib.scrypt` y `hmac` (estándar)	Contraseñas con sal y token de sesión	Decisión del grupo: visitante registrado	Seguridad razonable sin sumar librerías.

Tecnología	Para qué se usa	Origen	Qué aporta / costo
PostgreSQL 16 + `psycopg` 3 (SQL plano)	Guardar inscripciones, cupos y usuarios	Decisión del grupo	La consigna no pide base de datos, pero el **cupo es un estado compartido** y la reserva atómica (D5) solo se puede probar contra una base real; visible en DBeaver. Costo: requiere Docker (las pruebas unitarias corren sin él).
Docker Compose	Levantar PostgreSQL con un comando	Decisión del grupo	Cualquier integrante reproduce la base igual; reemplaza instalar PostgreSQL a mano.
FastAPI + uvicorn	API HTTP entre el front y las reglas	Decisión del grupo	Valida y documenta (`/docs`) y se prueba sin servidor con `TestClient` . Costo: una capa más; se justifica porque el front está separado.
React 19 + Vite 8	Pantallas de la inscripción	**PO**: paleta y tipografía del front; la US es de interfaz (elegir, completar, aceptar)	Permite probar el recorrido completo del visitante; aporta 110 pruebas. Costo: más código fuera del núcleo evaluado (TDD del dominio).
Vitest + React Testing Library	Pruebas del front	Decisión del grupo (reemplaza a Jest)	Misma API que Jest, integrado a Vite; prueba lo que ve el usuario, no la implementación.
`liquid-glass-react`	Efecto *Liquid Glass* de los paneles	**Exigencia de estilo del front**	Encerrada en un solo componente (`Vidrio.jsx`): si falla, se cambia un archivo.
`@fontsource/lato`	Tipografía Lato	**PO**: la fuente debe ser Lato	Se sirve desde el propio proyecto: sin depender de un CDN externo ni mandar datos a terceros.
Git / GitHub	Historial rojo-verde-refactor y *pull requests*	Consigna: se evalúa el ciclo	Es la evidencia verificable del proceso.

Conclusión. Lo estrictamente necesario para la consigna es Python, pytest, el estilo PEP 8 y este documento. El resto se agregó porque la User Story o el PO lo piden (QR, email real, Lato, paleta, *Liquid Glass*) o porque hacen verificable una regla que si no quedaría sin probar (la base de datos para el cupo atómico). Se mantuvieron pocas dependencias: **4 librerías de ejecución en Python** (`qrcode` , `psycopg` , `fastapi` , `uvicorn`) y **4 en el front** (`react` , `react-dom` , `liquid-glass-react` , `@fontsource/lato`). No se usó ningún ORM, ni *framework* de CSS, ni biblioteca de componentes.

7. Trazabilidad con la User Story

Criterio de aceptación / prueba de usuario	Dónde se resuelve	Pruebas
Elegir una actividad de la lista **con cupo**	`Actividad` , `disponibilidad` , `GET /actividades`	`ACT-*` , `CAT-04..09` , `ACT-F*`
Elegir el horario entre los **disponibles** (parque abierto, actividad disponible)	`Parque` , `Horario` , turnos del día	`PAR-*` , `HOR-*` , `HOR-F*`
Indicar la **cantidad** de personas	`Inscripcion` (máximo 10 y ≤ cupo restante)	`INS-*` , `VIS-F*`

Criterio de aceptación / prueba de usuario	Dónde se resuelve	Pruebas
Datos de cada visitante (nombre, DNI, edad, **talla si la actividad la pide**)	`Visitante`	`VIS-01..18`, `VAL-F*`
Aceptar los **T&C de la actividad**	`TerminosYCondiciones`, `terminos_texto`	`INS-*`, `TYC-*`, `CONF-F*`
Email con código QR al finalizar	`mensaje_email`, `qr`, SMTP	`QR-*`, `EML-*`, `SMTP-*`, `SRV-*`
Pasa: inscripción completa con email	`InscripcionService`	`ACE-01`
Falla: sin cupo en el horario	`Actividad`, repositorio	`ACE-02`, `REP-05`
Pasa: sin talla en una actividad que no la requiere	`Visitante`	`ACE-03`, `VIS-*`
Falla: parque cerrado o sin turno	`Parque`, `Horario`	`ACE-04`, `PAR-*`
Falla: sin aceptar los T&C	`Inscripcion`	`ACE-05`
Falla: sin la talla requerida	`Visitante`	`ACE-06`
Falla: T&C de otra actividad / sin haber iniciado sesión	`Inscripcion`	`ACE-07`, `ACE-08`

8. Estrategia de pruebas

8.1 Pirámide

- **Base (345 + 110):** unitarias del dominio, del servicio con dobles y de las pantallas con API falsa. Corren en segundos, sin infraestructura.
- **Medio (35):** contratos contra PostgreSQL real y `qrcode` real, incluida la carrera por el último cupo.
- **Cima:** aceptación (`ACE-01..08`, que además de las pruebas de usuario de la consigna cubren T&C ajenos y falta de sesión), cada una ejecutada con dobles **y** con PostgreSQL + QR real.
- **Externa (1):** un email real con QR a una casilla del grupo; requiere las credenciales de Gmail en el `.env`.

8.2 Contratos que se ejecutan dos veces

Repositorio de inscripciones, repositorio de usuarios y generador de QR comparten pruebas entre el doble y la pieza real. Es la garantía de que reemplazar un doble no cambia el comportamiento.

8.3 Hallazgos que encontraron las pruebas y las corridas manuales

#	Hallazgo	Quién lo encontró	Arreglo
1	El fake reservaba el cupo sobre una copia de la actividad	prueba de contrato `REP-03`	reservar sobre la del repositorio
2	Una prueba dependía de que el fake devolviera la misma instancia	`REP-04`	preparar el estado solo con la API del repositorio
3	La API se colgaba con credenciales SMTP de ejemplo	corrida manual	tiempo límite de 10 s (`SMTP-06`)

#	Hallazgo	Quién lo encontró	Arreglo
4	Sin sesión, el rechazo revelaba qué DNI estaban inscriptos	corrida manual	`USUARIO_NO_REGISTRADO` temprano (`SRV-23`)
5	T&C de una actividad inexistente daba error 500	`SRV-30`	404
6	La API rechazaba al front abierto en `127.0.0.1` (CORS)	corrida manual en el navegador	orígenes configurables (`MAIN-05/06`)
7	El vidrio de la librería dibujaba un contorno interior en elementos anchos	corrida manual en el navegador	proporción máxima 1,6 (`VID-11/12`)
8	Una regla nueva (`INS-11`) rompió `INS-02` (actividad nula)	la suite completa	guarda en `errores_por_visitante`
9	La API ya iniciada no enviaba emails reales tras cargar el SMTP (lee el `.env` al arrancar)	corrida manual del grupo	reiniciar la API; quedó documentado en el README
10	Dos inscripciones distintas se veían como un solo email (mismo asunto)	corrida manual del grupo	asunto único con código, fecha y horario (`EML-19/20`)

9. Límites y trabajo futuro

Tema	Límite actual	Mejora posible
Caída entre guardar y enviar el email	La inscripción queda guardada sin email (D6)	Estado *pendiente* + reintentos en segundo plano
Cancelaciones	No hay cancelar ni modificar una inscripción (no está en la US)	Nueva US
Sesión	Token en `localStorage`, sin renovación	Cookie `HttpOnly` y refresco
Contraseña	Mínimo de 6 caracteres (*provisorio*)	Definir la política con el PO
Menores de edad	Se pide la edad pero no hay un adulto responsable	Definirlo con el PO (P10b)
Refracción del vidrio	Solo en navegadores Chromium	Evaluar otra técnica
Preguntas de baja prioridad	P10b, P12b, P15b, P18, P19b, P20, P21 siguen abiertas	Resolver con el PO

10. Cómo ejecutarlo

Los pasos completos están en `codigo/README.md`. En resumen:

```
cd codigo
docker compose up -d                      # PostgreSQL
cd backend
PYTHONPATH=src .venv/Scripts/python.exe -m ecoharmony.preparar_base
.venv/Scripts/python.exe -m pytest        # 380 pruebas (las de base se saltean si no hay
PostgreSQL)
.venv/Scripts/python.exe -m ruff check .  # PEP 8
cd ../frontend
npm install && npm test                   # 110 pruebas
```

11. Bibliografía

- Beck, K. *Test-Driven Development by Example*. Addison-Wesley, 2002. Patrones: *Starter Test*, *One Step Test*, *Fake It*, *Triangulate*, *Obvious Implementation*, *Child Test*, *Mock Object*, *Crash Test Dummy*, *Clean Check-in*.
- Ble, C. *Diseño Ágil con TDD*. Caps. 3 (ATDD), 6 (dobles de prueba), 8–9 (proyecto de ejemplo).
- PEP 8 – *Style Guide for Python Code* (estilo elegido por el grupo en el TP2).
- Fuentes consultadas para redactar las normas de seguridad de los términos y condiciones (no existe una norma única; los parques fijan sus requisitos): American Academy of Pediatrics, *Zip Line Safety Tips* (healthychildren.org); reglamentos de rocódromos de la Universidad de Extremadura, de Salamanca y de Vélez-Málaga (equipo homologado, casco, arnés y menores acompañados); normas de visita de Bioparc Valencia, Bioparco (Roma) y Night Safari de Singapur (no alimentar, no tocar, sin flash, mantener la distancia); normativas de seguridad de talleres de herramientas y guías de seguridad en jardinería (calzado cerrado, cabello recogido, inspección de herramientas).
- Modelo de términos y condiciones de participación aportado por el grupo, adaptado a cada actividad.
- Enunciado del TP2 (EcoHarmony Park) y respuestas del Product Owner (ver preguntas al PO).